A Hybrid Statistical and Machine Learning AIOps Framework for Infrastructure Monitoring and Incident Correlation

Nam.G.D

Devotion, Germany, gnam1819@gmail.com

Abstract. This paper presents a hybrid AIOps framework that combines statistical analysis, machine learning, event correlation, and incident generation for infrastructure monitoring. The framework is deployed on a multi-node VPS environment supporting a quantitative trading system, where operational metrics and logs are collected from real running services such as Apache Airflow. Statistical methods including Z-score, EWMA, and Rate of Change are combined with the Isolation Forest algorithm to identify abnormal infrastructure behavior. Detected metric and log events are stored in an operational data store and correlated into higher-level incidents based on host and temporal context. Experimental results show that the hybrid approach can identify complementary anomaly patterns across CPU and memory signals, while event correlation reduces fragmented operational events into incident-level information. The evaluation also highlights an important limitation: temporal and host correlation alone do not necessarily imply causality. The proposed framework therefore provides a practical foundation for infrastructure monitoring, incident correlation, and operator-oriented AIOps analysis.

Keywords: AIOps · Statistical Methods · Machine Learning · Isolation Forest · Infrastructure Monitoring · Event Correlation · Incident Generation

I. INTRODUCTION

Modern infrastructure continuously generates vast amounts of operational data, such as metrics, logs, and system events. As the number of servers and services grows, monitoring based on dashboards and static thresholds becomes inefficient; such methods struggle to adapt to workload fluctuations and tend to generate a flood of disjointed alerts.

Drawing on three years of experience in DevOps, I have researched the development of an AIOps framework that combines statistical methods and machine learning to analyze infrastructure monitoring data. Techniques such as Z-score, EWMA, Seasonal Baseline, and Isolation Forest are employed to detect anomalies within the operational data. The resulting findings are normalized into operational events and stored in an AIOps Operational Data Store.

These operational events are then processed by an event correlation mechanism that groups related events based on server, service, and time, thereby forming operational-level incidents. This approach reduces the volume of isolated events, facilitates clearer problem identification, and provides better context for incident resolution.

The system delivers practical value to infrastructure operations teams, including system administrators, DevOps engineers, SREs, and NOC staff. Instead of manually checking multiple data sources, operators can utilize aggregated incidents to rapidly pinpoint problem areas, prioritize remediation efforts, and reduce the time spent on incident analysis. Furthermore, the system establishes a foundation for integrating AI-based Root Cause Analysis (RCA) and AIOps agents to assist with diagnostics, recommendations, and interaction with operational data.

The key contributions of the research include:

- Combining statistical methods and machine learning for infrastructure data analysis.
- Developing mechanisms for event correlation and incident generation.
- Implementing and evaluating the framework in a multi-server environment.
- Providing a platform to support infrastructure operations and integrate AI-based RCA components and AIOps agents.

II. RELATED WORK

Current research on AIOps primarily focuses on three main areas: analyzing infrastructure data using statistical and machine learning methods, correlating events to form incidents, and leveraging AI/LLMs to support operational analysis.

In infrastructure monitoring, statistical techniques such as Z-score, EWMA, and Seasonal Baseline are commonly employed to detect deviations from normal system behavior. Additionally, unsupervised machine learning algorithms like Isolation Forest enable the identification of anomalous observations without the need for a pre-defined set of anomaly labels. These methods are well-suited for time-varying metric data and are frequently applied to the monitoring of CPU, memory, disk, network, and container resources.

However, events generated from individual metrics or logs can still lead to alert fragmentation. Consequently, event correlation is used to group related events based on attributes such as host, service, and timestamp. This process yields incidents with greater operational significance, thereby reducing the volume of discrete signals and providing better context for investigation.

Another area of research involves integrating AI-based Root Cause Analysis (RCA) and LLM-based AIOps agents to assist in explaining root causes, querying operational data, and generating recommendations in natural language. In this specific study, however, the AI RCA and LLM components primarily serve downstream functions to demonstrate end-to-end integration capabilities, while the core focus remains on statistical and machine learning analysis, as well as event correlation and incident generation.

Unlike approaches that focus solely on isolated steps, this study integrates statistical methods, machine learning, and event correlation within a unified AIOps framework; it transforms monitoring data into operational events and subsequently aggregates them into incidents to support infrastructure operations.

III. PROPOSED ARCHITECTURE

The proposed architecture follows a multi-stage pipeline model in which operational data is collected from the infrastructure, processed using statistical and machine learning methods, and stored as operational events; these events are then correlated to form incidents. These incidents are subsequently utilized by AI RCA components and AIOps Agents to facilitate analysis and interaction with operators.

The system's overall processing workflow is described as follows:

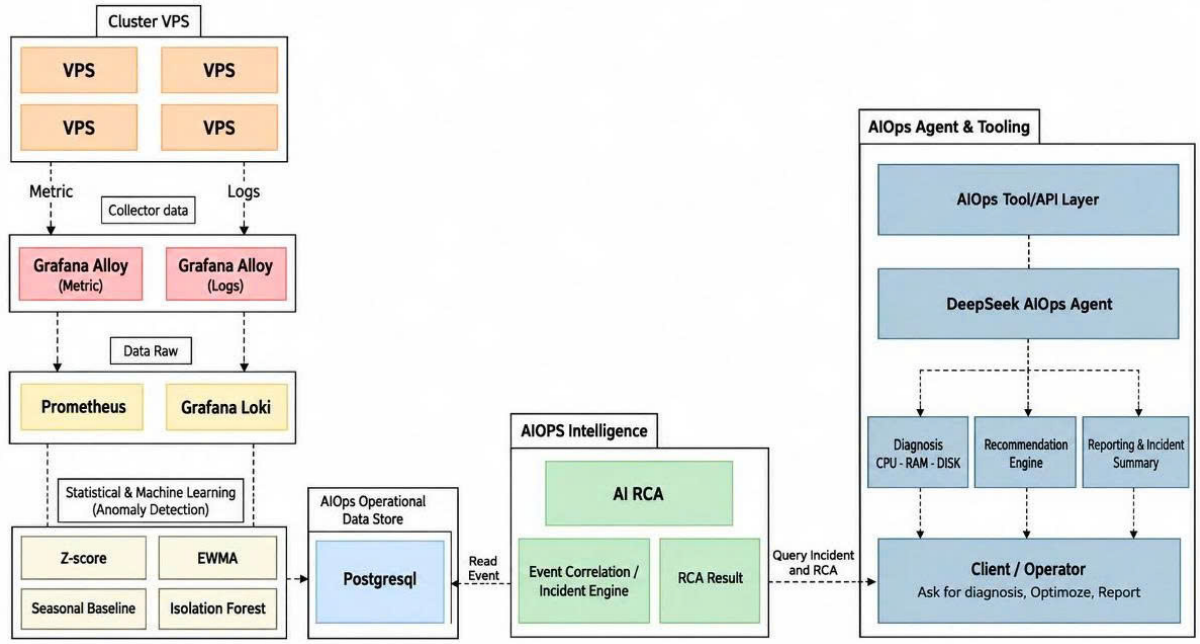


Figure 1: Overall architecture of the proposed hybrid AIOps framework.

The **Data Collection and Observability** component is responsible for gathering metrics and logs from VPS instances. Subsequently, the **Statistical and Machine Learning Analysis** module employs methods such as Z-score, EWMA, Seasonal Baseline, and Isolation Forest to detect anomalous behaviors and generate operational events. These events are stored in the **AIOps Operational Data Store** and processed by the **Event Correlation and Incident Generation** component, which groups related events into incidents.

In the backend layers, **AI-based RCA** utilizes incidents and related evidence to analyze root causes, while the **AIOps Tool/API Layer** provides data access interfaces and AIOps functionalities to the **DeepSeek AIOps Agent**. The Agent serves as the interface for interacting with operators, facilitating diagnostics, recommendations, and report generation.

The research focus of this architecture centers on two primary components: **Statistical and Machine Learning Analysis**, and **Event Correlation and Incident Generation**. The AI RCA, Tool/API, and DeepSeek AIOps Agent serve as downstream components to complete and demonstrate the end-to-end AIOps process.

A. Data Collection and Observability

The system uses Grafana Alloy to collect operational data from VPS instances. The two primary data types are metrics and logs. Metrics reflect resource status—such as CPU, memory, disk, network, and containers—while logs provide information on the activity of the operating system, services, and applications.

Metrics are forwarded to Prometheus for time-series storage, whereas logs are sent to Grafana Loki to facilitate querying and analysis. This layer is responsible solely for data collection and providing input telemetry for subsequent processing stages.

The data flow can be represented as follows:

$$\text{VPS} \rightarrow \text{Grafana Alloy} \rightarrow \text{Prometheus / Grafana Loki}$$

B. Statistical and Machine Learning Analysis and AIOps Operational Data Store

1. Statistical Analysis

Statistical analysis is used to identify abnormal changes in infrastructure metrics by comparing current observations with historical behavior. In the proposed AIOps framework, three statistical methods are applied: Z-score, Exponentially Weighted Moving Average (EWMA), and Rate of Change.

The Z-score measures how far the current observation deviates from the historical mean:

$$z_t = \frac{x_t - \mu}{\sigma}$$

Where x_t is the observed value at time t, μ is the historical mean, and σ is the standard deviation. A large absolute Z-score indicates that the current observation is significantly different from its historical distribution.

The EWMA method is used to track gradual changes and recent trends in the metric series:

$$s_t = \alpha x_t + (1 - \alpha) s_{t-1}$$

Where x_t is the current observation, s_t is the smoothed value, and α is the smoothing factor. EWMA gives greater importance to recent observations and is therefore useful for identifying gradual shifts in system behavior.

The Rate of Change measures the relative variation between consecutive observations:

$$\text{ROC}_t = \frac{x_t - x_{t-1}}{x_{t-1}}$$

This method is useful for detecting sudden increases or decreases in infrastructure metrics such as CPU usage, memory consumption, and network traffic.

These statistical methods are executed independently on the same metric window. Their results are then combined to determine whether the current observation represents abnormal operational behavior.

For example, a sudden CPU variation may produce a high Z-score, while a gradual increase in memory consumption may be identified by EWMA. Rate of Change can additionally detect rapid transitions between consecutive samples. Combining the three methods provides multiple statistical perspectives on the same infrastructure behavior.

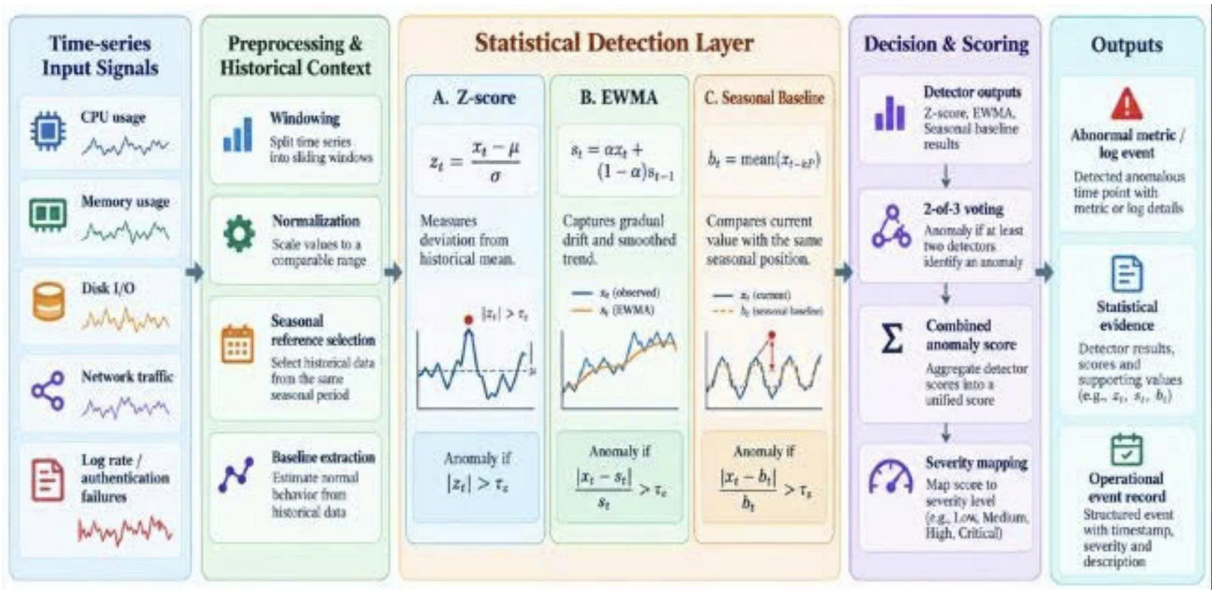


Figure 2: Statistical Analysis Model for AIOps Infrastructure Monitoring.

2. Machine Learning Analysis

Machine learning analysis is introduced to complement the statistical detectors by identifying unusual metric patterns that may not be fully captured by fixed statistical thresholds. In the proposed AIOps framework, Isolation Forest is used as the machine learning method.

Isolation Forest is an unsupervised learning algorithm that detects abnormal observations by isolating them through a set of randomly generated isolation trees. Instead of modeling normal behavior directly, the algorithm assumes that anomalous observations are easier to separate from the majority of the data.

For each analysis window, infrastructure metric samples are transformed into feature vectors and processed by multiple isolation trees. At each tree, a feature and split value are selected randomly. Observations that can be isolated using only a small number of splits obtain shorter path lengths, while normal observations usually require deeper traversal through the tree.

The anomaly score can be expressed as:

$$s(x, n) = 2^{-\frac{E[h(x)]}{c(n)}}$$

Where $E[h(x)]$ is the average path length of observation x across the isolation trees, and $c(n)$ is a normalization factor based on the number of samples n.

A shorter average path length results in a higher anomaly score, indicating that the observation is easier to isolate and is therefore more likely to be abnormal. In contrast, a longer path length produces a lower anomaly score and indicates behavior closer to the normal data distribution.

In the proposed system, Isolation Forest is applied to infrastructure metrics such as CPU usage, memory usage, load average, and other numerical operational signals. Its output is used together with the statistical detector results to provide additional evidence when deciding whether an operational event should be generated.

For example, a memory observation that is unusual relative to the distribution of previous samples may be classified as anomalous by Isolation Forest even when not all statistical detectors produce the same decision. This provides an additional machine learning perspective for identifying abnormal infrastructure behavior.

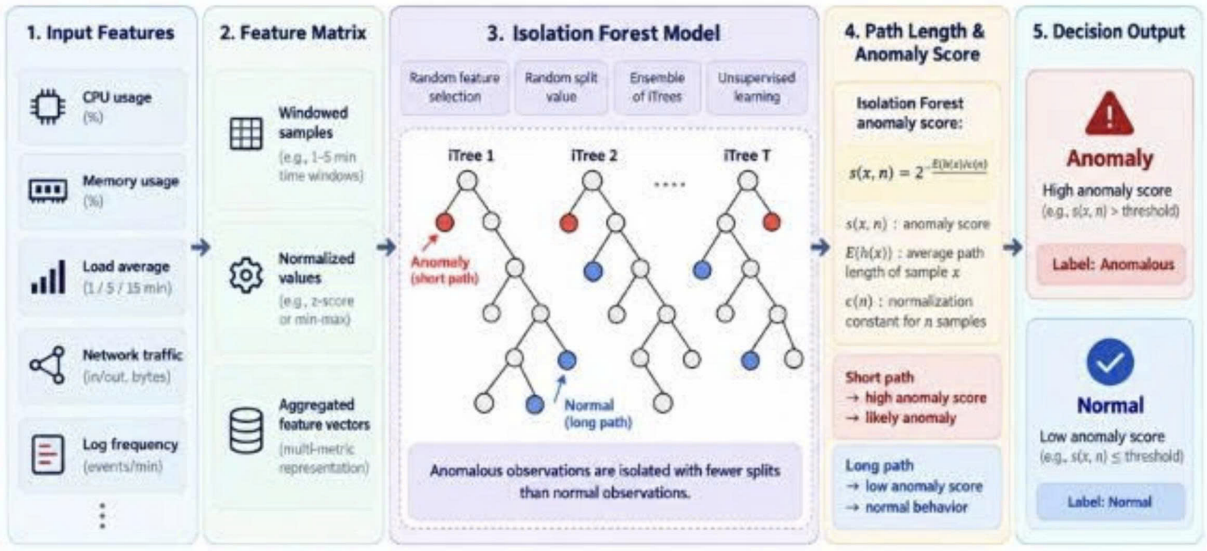


Figure 3: Isolation Forest-based Machine Learning Analysis Model for AIOps Infrastructure Monitoring.

In the current implementation, Isolation Forest primarily contributes a boolean anomaly decision to the AIOps pipeline. The result is combined with Z-score, EWMA, and Rate of Change outputs in the detector aggregation stage before the final anomaly decision and operational event generation.

Thus, the role of machine learning in the proposed framework is not to replace statistical analysis, but to provide an additional detection mechanism that improves the robustness of infrastructure monitoring by capturing observations that exhibit unusual distributional behavior.

3. AIOps Operational Data Store

Once created, operational events are stored in PostgreSQL, which serves as the system's AIOps Operational Data Store.

Unlike Prometheus and Grafana Loki, PostgreSQL does not store raw telemetry; instead, it stores operational data that has been processed and standardized to support event correlation, incident generation, and downstream components.

Key tables include: Key tables include:

Table	Function
Events	Store operational events generated from metrics and logs
Incidents	Store incidents formed through the correlation process
Rca_results	Store the results of incident root cause analysis

A typical event includes attributes such as:

- `event_id`
- `host`
- `service`
- `event_type`
- `severity`
- `timestamp`
- `score`
- `raw_data`

C. Event Correlation and Incident Generation

Once operational events are created and stored in the AIOps Operational Data Store, the system performs event correlation to identify events likely associated with the same operational issue. This is the second core component of the study, as a real-world incident typically generates multiple distinct signals rather than just a single, isolated event.

The correlation process utilizes key attributes such as host, service, event type, severity, and timestamp. Events occurring on—or related to—the same infrastructure component within a close timeframe are evaluated for grouping into a single incident.

The temporal correlation condition can be simply expressed as:

$$|t_i - t_j| \leq \Delta T$$

Where t_i and t_j are the timestamps of two events, and ΔT is the time window used for the correlation process.

For example, the following can coexist on the same VPS:

$$\begin{gathered}\text{CPU utilization deviation} \\ + \\ \text{Network traffic deviation} \\ + \\ \text{Authentication failure} \\ \downarrow \\ \text{Correlated Incident}\end{gathered}$$

In this case, each individual event describes only one manifestation of the system. When correlated based on the same host and relevant time frame, they are aggregated into an incident with richer context.

Each incident stores key information such as:

- Incident ID
- Host
- Service
- Severity
- First event time
- Last event time
- Number of related events
- Related event information

The processing flow can be represented as follows:

$$\text{Operational Events} \rightarrow \text{Event Correlation} \rightarrow \text{Incident Generation} \rightarrow \text{Incident}$$

D. AI-based RCA and AIOps Agent Interaction

Once an incident is created, the system employs AI-based Root Cause Analysis (RCA) to analyze the potential cause of the issue. This component utilizes incident data and relevant evidence—such as metric events, log events, host, service, severity, and timestamps—to generate the root cause analysis results.

RCA results may include:

- Root cause
- Confidence score
- Supporting evidence
- Affected component
- Recommended action

These results are stored for use in subsequent query steps. In this study, the AI-based RCA serves as a downstream component to illustrate how the incident is utilized after its generation, rather than being the primary subject of evaluation.

To connect backend components with operators, the system employs an AIOps Tool/API Layer. This layer provides controlled access interfaces for metrics, logs, incidents, and RCA results.

Instead of directly accessing storage systems, the DeepSeek AIOps Agent invokes the corresponding tools to retrieve the necessary data.

The interaction flow can be described as follows:

Incident → AI-based RCA → RCA Result → AIOps Tool/API Layer → DeepSeek AIOps Agent → Operator

The DeepSeek AIOps Agent serves as a natural language interface. Operators can ask questions regarding CPU and memory status, recent logs, ongoing incidents, or the root cause of a specific incident. The agent selects the appropriate tool, retrieves the data, and synthesizes it into an easy-to-understand response.

Example:

Operator:
"Why is node4 having a critical incident?"
↓
DeepSeek AIOps Agent
↓
Query Incident
↓
Get RCA Result
↓
Generate Explanation
↓
Return Diagnosis and Recommendation

In the proposed architecture, these four components have the following primary functions:

- **AI-based RCA:** analyzes incident root causes based on evidence
- **AIOps Tool/API Layer:** provides interfaces for data access and AIOps functions
- **DeepSeek AIOps Agent:** orchestrates tools and generates natural language responses
- **Operator Interaction:** supports operators in inspection, diagnosis, receiving recommendations, and reporting

IV. EXPERIMENTAL EVALUATION

A. Experimental Environment

The proposed AIOps framework is deployed directly on the infrastructure of a quantitative trading system rather than in an isolated synthetic environment. Metrics and logs used for statistical and

machine learning analysis are collected from real running services and workloads within this platform, including Apache Airflow components and other supporting infrastructure services. Controlled stress scenarios are introduced only to create identifiable abnormal conditions for experimental evaluation, while the underlying telemetry is generated by the operational system itself.

The experimental environment is deployed across a cluster of multiple VPS instances, with two representative nodes selected to illustrate the deployment architecture. Node 2 serves as the monitoring node, responsible for running Prometheus and Grafana Loki to ingest and store telemetry data. Node 3 functions as the AIOps node, hosting the AIOps Operational Data Store along with components for event processing, incident management, and root cause analysis (RCA).

Node2 is configured with 2 CPU cores, 11 GiB of RAM, and 96 GB of disk space, running Ubuntu 24.04.3 LTS. Grafana Alloy is deployed to collect metrics and logs from the infrastructure. At the time of the experiment, Alloy uses the `grafana/alloy:v1.19.2` image, while the monitoring backend consists of Prometheus and Grafana Loki.

Component	Experimental configuration
Monitoring Node	Node2
CPU	2 cores
Memory	11 GiB
Disk	96 GB
Operating System	Ubuntu 24.04.3 LTS
Telemetry Collector	Grafana Alloy
Metric Backend	Prometheus
Log Backend	Grafana Loki
AIOps Node	Node3
Operational Data Store	PostgreSQL
Statistical Methods	Z-score, EWMA, Rate of Change
Machine Learning	Isolation Forest

Prometheus is queried via its API, returning time-series data for the `up` metric, which encompasses the system's monitoring targets. This result confirms that telemetry from these targets has been successfully ingested by Prometheus and is available for querying to support statistical analysis and machine learning tasks.

Grafana Loki is used to store and query log data. Results from the Loki API indicate that various labels have been collected—including `host`, `hostname`, `node`, `service_name`, `level`, `log_type`, `container`, and `syslog_identifier`. These labels provide the necessary information to categorize logs by node, service, and event type during analysis.

On node3, PostgreSQL is used as the AIOps Operational Data Store. The database contains tables that support the entire AIOps pipeline, including:

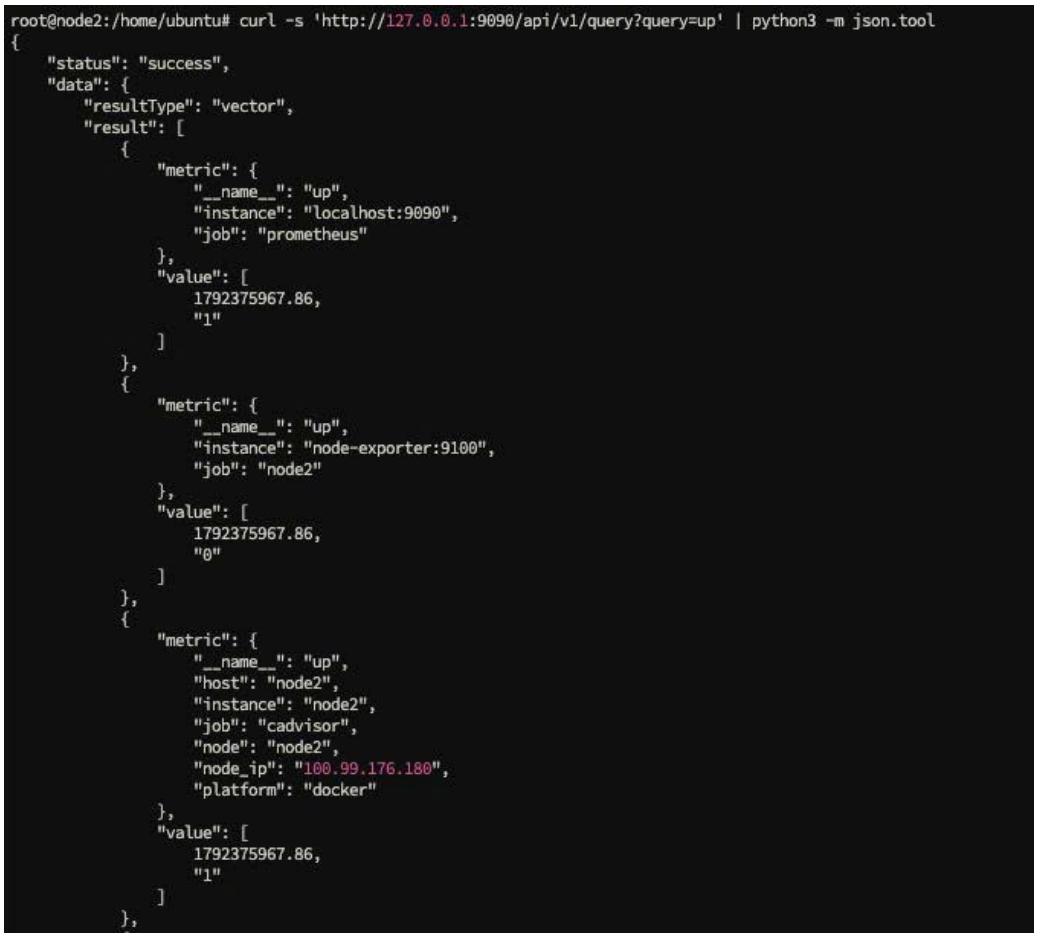


Figure 4: Prometheus telemetry query in the experimental environment.



Figure 5: Log labels available in Grafana Loki during the experiment.

Table	Purpose
Events	Stores processed operational events
Incident_events	Maintains relationships between events and incidents
Incidents	Stores generated incidents
Rca_results	Stores root cause analysis results
remediation_actions	Stores remediation-related actions
Reports	Stores generated reports
Tickets	Stores operational ticket information

Within the scope of this study, the three most important data groups are events, incidents, and incident-event relationships, as they directly support statistical and machine learning analysis, as well as event correlation and incident generation.

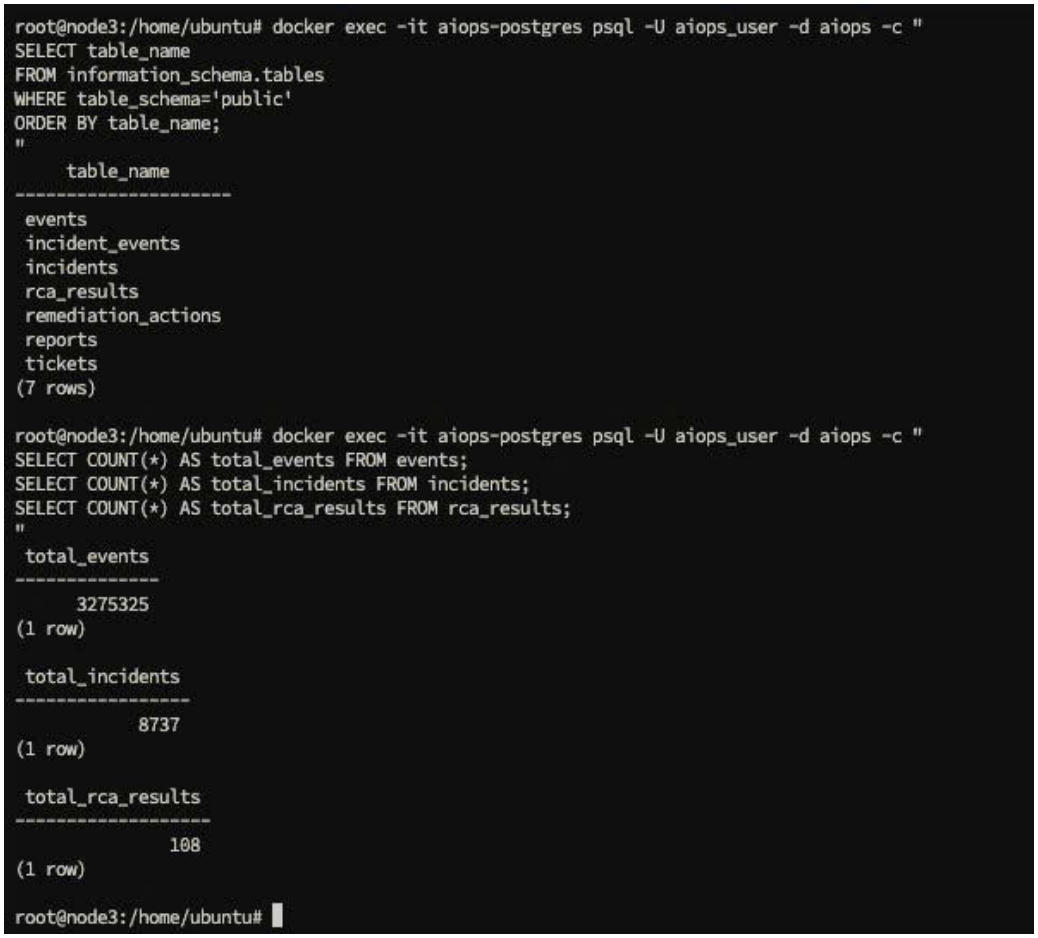


Figure 6: Main tables of the AIOps Operational Data Store.

At the time of empirical data collection, PostgreSQL contained:

Data type	Number of records
Operational Events	2,553,498
Incidents	7,375
RCA Results	40

These figures indicate that the pipeline generated and stored a large volume of operational event data prior to performing correlation to create incidents.

The environment described above provides a sufficiently large volume of real-world data to evaluate the two core components of the research: Statistical and Machine Learning Analysis, and Event Correlation and Incident Generation.

B. Experimental Scenarios

Experimental scenarios were conducted on a monitored VPS to simulate various operating conditions and evaluate the system's ability to detect operational events, correlate these events, and identify incidents. Four primary scenarios were employed: CPU stress, memory stress, authentication failure, and multi-signal correlation.

1. CPU Resource Stress

A CPU stress scenario generates a high processing load over a specified duration, intentionally causing significant changes in CPU-related metrics and system load. The objective of this test is to evaluate the effectiveness of statistical and machine learning methods in detecting deviations from normal operating conditions.

2. Memory Resource Stress

The memory stress scenario increases memory usage on the test node. This test is used to observe the ability to detect resource changes over time, particularly for methods that rely on baselines and historical data.

3. Authentication Failure

The authentication failure scenario involves multiple unsuccessful SSH authentication attempts. Unlike the first two scenarios, which rely on metric data, this test generates log events to evaluate the system's ability to collect, analyze, and transform anomalous logs into operational events.

4. Multi-signal Correlation

A multi-signal scenario generates multiple types of signals simultaneously on the same node, including resource behavior and authentication-related events. The objective is to evaluate the capability of Event Correlation to identify events occurring within the same timeframe and aggregate them into an incident with richer context.

The four scenarios can be summarized as follows:

Scenario	Generated behavior	Main purpose
CPU Resource Stress	High CPU and load behavior	Statistical and ML analysis
Memory Resource Stress	Increased memory usage	Resource behavior analysis
Authentication Failure	Repeated SSH authentication failures	Log-based event analysis
Multi-signal Correlation	Multiple metric and log events	Event correlation and incident generation

5. End-to-End AIOps Agent Validation

After completing the four scenarios, the DeepSeek AIOps Agent is used to re-query the experimental time period on node4. The agent retrieves operational events, correlation records, incident information, and supporting evidence via the AIOps Tool/API Layer.

The final validation is designed to verify that the outputs produced by the statistical and machine learning analysis, operational event generation, and event correlation stages can be retrieved and presented through the AIOps Tool/API Layer and DeepSeek AIOps Agent.

The validation process starts with the experimental scenarios, whose telemetry is analyzed by the statistical and machine learning modules. The detected results are converted into operational events, correlated into incidents, and then exposed through the AIOps Tool/API Layer. Finally, the DeepSeek AIOps Agent retrieves and presents the resulting evidence to the operator.

The following response was returned by the AIOps Agent after querying the experimental period on node4. It summarizes the detected statistical and machine learning results, operational events, correlated incidents, and supporting evidence generated during the experimental scenarios.

Real-world example:

Operator prompt

> Summarize node4 AIOps status for today using live data only. Show the main CPU, memory, and authentication anomalies, including Z-score, EWMA, Rate of Change, Isolation Forest when available, and anomaly scores. Then summarize the correlated incidents, clearly distinguishing time/host correlation from causal evidence. Use +07:00 timestamps and ignore older workspace reports.

AI-generated response — verbatim transcript *The following text and tables reproduce the Agent's response. Claims and confidence values within this transcript are reported by the Agent.*

node4 (**node4-coldstore**) — AIOps status, 2026-10-15 (+07:00)

1. Current state (latest live samples)

Signal	Value	Assessment
CPU	47.3–50.4% across samples 21:43–21:45	Normal, no host-level breach
Memory	32.4–36.2%	Normal
Root disk /dev/sda1 (/, ext4)	30.11%	Normal
Network	RX ∼25–27 KB/s, TX ∼43–44 KB/s	Normal
Top container CPU	airflow-worker ∼35%, airflow-dag-processor ∼23%, airflow-scheduler ∼13%, airflow-triggerer ∼12%, alloy-node4 ∼8%	Airflow-driven load
Top container mem	airflow-worker ∼1.55–2.01 GB, airflow-apiserver ∼368 MB, airflow-scheduler ∼283 MB	—

Important: no host-level CPU or memory anomaly ever fired today. Every CPU/memory anomaly below is container-level, and the newest host reading is healthy. The elevated host CPU (≈50%) is normal Airflow workload, not an alert.

2. Main CPU anomalies (container-level)

Time (+07:00)	Metric / container	Observed	Baseline	Z-score	EWMA	Rate of Change	Isolation Forest	Anomaly score
21:41:00	container_cpu_percent (unidentified)	22.4499%	n/a	n/a	n/a	n/a	n/a	0.802 (live alert 373675a7)
18:25:59	container_cpu_percent airflow-apiserver	0.5927	0.9318	3.92	is_anomaly = false	is_anomaly = false	true	—
15:45:06	container_cpu_percent airflow-apiserver	0.2539	0.6757	—	dev 0.6242	−0.64	—	0.9392
15:30:12	container_cpu_percent airflow-apiserver	0.1917	0.7037	10.25	fired	fired	—	1.0
15:25:04	container_cpu_percent airflow-apiserver	0.305	0.652	—	—	−0.581	—	0.9112
15:15:10	container_cpu_percent airflow-apiserver	0.1975	0.7048	9.94	fired	fired	—	1.0
14:55 / 15:00	container_cpu_percent airflow-apiserver	0.3561 / 0.3518	—	—	—	—	—	0.857 / 0.8612
14:30:05	container_cpu_percent airflow-apiserver	0.3419	0.6656	—	score 1.0	−0.5235	—	0.8644
14:25:06	container_cpu_percent airflow-apiserver	0.3403	0.6659	—	—	−0.5377	—	0.8654
14:20:05	container_cpu_percent airflow-apiserver	0.2554	0.6819	—	—	dev 0.6255	—	0.9199

Pattern: the dominant CPU signal is a repeated drop in airflow-apiserver CPU against its ∼0.65–0.70 baseline (Z-scores ∼10 at 15:15 and 15:30), plus a concurrent network_rx_bytes_per_second drop. Many of these are downward anomalies — inflated scores with no service impact.

3. Main memory anomalies (container-level)

Time (+07:00)	Container	Observed	Baseline	Z-score	EWMA	Rate of Change	Isolation Forest	Anomaly score
21:40:59	unidentified (container_memory_bytes)	380,436,480 B (∼363 MiB)	n/a	n/a	n/a	n/a	n/a	0.898 (live alert 4a2faff0)
∼16:16–16:25	airflow-worker (container_memory_working_set_bytes, apache/airflow:3.3.1, cadvisor, step 60s, window 60 min)	3,656,699,904 B (∼3.66 GB)	1,610,851,601 B (∼1.61 GB)	10.4397	fired	fired	fired	0.9317
17:36:03	airflow-scheduler	403,845,120 B (∼403.8 MB)	∼290.6 MB	4.4634	—	—	true	0.8093
15:45:04	airflow-dag-processor	339,681,280 B (∼339.7 MB)	221,030,060 B	3.8636	dev 0.5368	+0.5388 (+53.7%)	—	1.0

The airflow-worker event is the only anomaly today where all four requested detectors fired (ewma, zscore, rate_of_change, isolation_forest) — and it is the single most significant metric anomaly of the day: +127% memory working-set growth, Z-score 10.44.

4. Authentication anomalies (the day's dominant signal)

Field	Value
Type	authentication_failure (log_anomaly), service ssh_service / sshd
Severity	Warning
Anomaly score	0.85 per event (log anomalies carry no Z/EWMA/ROC/IF fields)
Evidence	Raw journal: Failed password for root from 77.239.124.68 port . . . ssh2, Invalid user debian from . . . , pam_unix(sshd:auth): authentication failure . . . rhost=. . .
Successful compromise	No evidence found in retrieved events

Roughly 75–80+ failed events per burst, spanning at least 12–18 distinct external IPs — a sustained brute-force / credential-stuffing campaign, not normal traffic.

5. Detector-field availability (what actually exists)

Detector	Availability today
Z-score	Numeric: 10.44, 10.25, 9.94, 4.46, 3.92, 3.86, 3.67, 3.46, 3.0, 1.39
Rate of Change	Numeric where captured: -0.74 to $+0.54$ (e.g. -0.6806, -0.6563, -0.581, $+0.5388$)
EWMA	Partial — mostly score 1.0 or boolean is_anomaly; numeric deviation only on a few events (0.7241, 0.6242, 0.5368)
Isolation Forest	Boolean only (true/false / "all four detectors flagged") — no numeric IF score exists anywhere
Anomaly score	Composite score present: 0.802–1.0

6. Correlated incidents — correlation vs. causation 20 open incidents returned (limit reached, so totals are minimums), all titled "Correlated incident on node4", correlation_score ≈ 0.7, spanning 14:30:05 $\rightarrow$ 21:16:00 (report query saw 50 spanning the same window, ∼25 critical / 25 warning). Services: predominantly docker, plus two critical snap_oracle-cloud-agent (19:55–20:10), plus ssh_service,

system, apt-daily-upgrade_service, session-5156_scope. All remain open. Two critical incidents had no RCA available.

Correlation type, explicitly

Link	Type	Verifiable?
All member events share host = node4 / node4-coldstore	Host correlation — the real join key	Yes (log evidence is host=node4-coldstore, unit=ssh.service)
Events fall within a rolling ∼2–10 min window	Time correlation — the only link between metric and log anomalies	Yes, but weak
Incident label docker / oracle-cloud-agent / apt-daily-upgrade vs. evidence service ssh_service	Component mismatch → host+time only	Label unsupported
SSH bursts typically 10–15 min before the incident's first event (15:55 → 16:10; 20:55 → 21:06)	Time proximity widens, not closes	Window mismatch

7. What has actual causal evidence

Claim	Causal evidence?
SSH brute-force / credential-stuffing against node4-coldstore caused the authentication-failure logs	YES — direct raw sshd/pam_unix evidence, tight bursts, external IPs, high username diversity. RCA confidence 82–88%
SSH activity caused the container CPU/memory anomalies	NO — time-correlation only. RCAs say "secondary symptom", "not proven", "cannot be confirmed", "correlation artifact", "no metric evidence provided". Metric-component confidence 45–72%

The cleanest statement of causation anywhere today is in RCA 58877013 (incident 66588b2b, confidence 45%): two independent anomaly classes, and verbatim "No evidence links the SSH authentication failures to the container memory anomaly." That incident pairs the day's strongest metric anomaly (airflow-worker 3.66 GB, Z = 10.44, all 4 detectors firing at ∼16:16) with an SSH burst at 16:05:43–16:06:03 — co-occurrence on the same host, explicitly not causation.

7. Gaps that limit today's picture

- 00:00–14:14 (+07:00): no telemetry returned for this window — it cannot be certified quiet, only unretrieved.
- Alert and incident lists are truncated (20 / 50 item limits) — counts are minimums, not totals.
- ∼13 warning incidents have no RCA; the two critical Oracle Cloud Agent incidents have none either.
- No numeric detector thresholds and no Isolation Forest scores exist in the platform's exposed payloads.
- Container for the live 21:41 CPU (22.45%) and 21:40 memory (380 MB) anomalies is not identified.
- Uptime is not exposed by this toolset.

8. Safe next actions (nothing was changed)

- Security first: block/rate-limit the attacking IPs (prioritise 77.239.124.68, 77.239.124.205, 109.160.32.x, 92.118.39.x); disable SSH password auth, set PermitRootLogin no, restrict SSH to a bastion/VPN, deploy fail2ban or sshguard.
- Forensics: audit sshd logs for any successful login from those IPs, plus new users, authorized_keys, cron, and unexpected outbound connections — RCAs repeatedly flag this as unverified.
- Investigate airflow-worker memory (3.66 GB vs. 1.61 GB, Z = 10.44) as an independent issue: worker concurrency, task memory limits, recent DAG changes; check for OOM kills/restarts around 16:01–16:25.
- Fix the evidence gap: make the metric-anomaly detector emit its Z-score / EWMA / ROC / IF values, thresholds and series into the incident payload — most incidents currently "anchor" on anomalies with no metric evidence.
- Tune correlation: require component match and tighten the window; the current host+time join with a 10–15 min log-to-metric offset is generating noisy "correlated incidents" and inflating critical counts.

Bottom line: node4-coldstore is resource-healthy right now (CPU ≈48–50%, memory ≈33%, disk 30%) with no host-level anomaly. The day's genuinely evidenced anomaly is an ongoing external SSH brute-force campaign (score 0.85, latest burst 21:45:29 +07:00, no compromise confirmed). The strongest metric anomaly is airflow-worker memory at 3.66 GB, Z = 10.44, all four detectors firing (∼16:16). The 20 open "correlated incidents" are joined by host identity plus a loose time window; only the SSH→auth-log link is causal — the SSH→CPU/memory and SSH→docker/oracle-cloud-agent links are time/host correlation without causal evidence.

End of AI-generated response

C. Evaluation Metrics

The evaluation focuses on two main aspects of the proposed framework: anomaly detection performance and event correlation quality. Since the experimental scenarios are intentionally generated, the corresponding test periods provide labeled abnormal windows that can be used as ground truth.

1. Precision

Precision measures the proportion of detected anomalies that correspond to actual abnormal conditions:

$$\text{Precision} = \frac{TP}{TP + FP}$$

Where TP represents correctly detected abnormal events and FP represents normal observations incorrectly classified as anomalous.

2. Recall

Recall measures the ability of the framework to detect abnormal conditions introduced during the experiments:

$$\text{Recall} = \frac{TP}{TP + FN}$$

Where FN represents abnormal conditions that occurred but were not detected.

3. F1-score

F1-score provides a balanced measure between Precision and Recall:

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

These three metrics are used to evaluate the overall effectiveness of the statistical and machine learning analysis pipeline.

4. Detector Agreement

Because the proposed framework combines multiple detection methods, the agreement between Z-score, EWMA, Rate of Change, and Isolation Forest is also examined.

For each metric event, the number of detectors identifying the observation as abnormal is recorded:

$$\text{Agreement} = \frac{N_{\text{flagged}}}{N_{\text{detectors}}}$$

Where $N_{\text{detectors}} = 4$.

For example, the strongest memory anomaly observed during the experiment was detected by all four methods, including Isolation Forest, while some CPU events were detected by only a subset of the detectors.

This metric helps evaluate whether statistical and machine learning techniques provide complementary or consistent anomaly evidence.

5. Composite Anomaly Score

The framework generates a combined anomaly score from the detector outputs. The observed scores in the experiment ranged approximately from 0.802 to 1.0 for reported metric anomalies.

Higher scores indicate stronger evidence that the observed behavior differs from the learned or historical baseline.

6. Event Correlation Quality

The evaluation focuses on four metrics that can be directly supported by the experimental results: **Scenario Detection Rate, Detector Agreement, Composite Anomaly Score, and Event Correlation Quality**.

6.1. Scenario Detection Rate

Scenario Detection Rate measures whether the framework successfully identifies the abnormal conditions intentionally introduced during the experiment.

$$\text{Detection Rate} = \frac{N_{\text{detected scenarios}}}{N_{\text{total scenarios}}}$$

Four experimental scenarios were used: CPU stress, memory stress, authentication failure, and multi-signal correlation. Since all four scenarios produced detectable operational events or correlated incidents, the experimental Scenario Detection Rate was:

$$\text{Detection Rate} = \frac{4}{4} = 100\%$$

This metric indicates whether the end-to-end AIOps pipeline can recognize each injected test condition.

6.2. Detector Agreement

The metric analysis pipeline combines four detectors: **Z-score, EWMA, Rate of Change, and Isolation Forest**. Detector Agreement measures how many of these methods identify the same observation as abnormal:

$$\text{Agreement} = \frac{N_{\text{flagged detectors}}}{N_{\text{total detectors}}}$$

where:

$$N_{\text{total detectors}} = 4$$

For example, the strongest memory anomaly observed on airflow-worker was detected by all four methods, resulting in:

$$\text{Agreement} = \frac{4}{4} = 100\%$$

The event showed memory usage of approximately 3.66 GB compared with a 1.61 GB baseline, with a Z-score of 10.4397, while EWMA, Rate of Change, and Isolation Forest also flagged the observation.

Other events showed only partial agreement. For example, one CPU event was detected by Z-score and Isolation Forest, while EWMA and Rate of Change did not classify it as anomalous.

This comparison helps determine whether the statistical and machine learning detectors provide consistent or complementary evidence.

6.3. Composite Anomaly Score

The framework combines detector outputs into a composite anomaly score representing the overall strength of the detected abnormal behavior.

During the experiment, reported metric anomaly scores ranged approximately from:

$$0.802 \leq \text{Score} \leq 1.0$$

Higher scores indicate stronger evidence that the observed behavior differs from its historical or statistical baseline.

The composite score is used together with the number of detectors that identify an anomaly before an operational event is generated.

6.4. Event Correlation Quality

Event Correlation Quality evaluates whether multiple operational events are grouped into meaningful incidents.

The correlation process is examined according to three aspects:

- Host consistency: whether correlated events belong to the same monitored node
- Temporal consistency: whether events occur within the configured correlation window
- Operational consistency: whether the grouped events represent the same operational problem or only occur close together in time

The experimental results show that many incidents were correlated using the same host and a rolling time window. However, this mechanism can also group independent signals such as authentication failures and container resource anomalies.

Therefore, correlation is evaluated separately from causality. A correlation is considered stronger when the grouped events share host, time, and operational context, while host and temporal proximity alone are treated as weaker evidence.

For example, the SSH authentication failures were directly supported by raw SSH log evidence, whereas no causal evidence showed that the SSH activity caused the CPU or memory anomalies.

Evaluation summary

Metric	Purpose
Scenario Detection Rate	Measures whether each injected experimental scenario is detected
Detector Agreement	Measures agreement among Z-score, EWMA, Rate of Change, and Isolation Forest
Composite Anomaly Score	Represents the overall strength of anomaly evidence
Event Correlation Quality	Evaluates whether related events are grouped into meaningful incidents

V. RESULTS AND DISCUSSION

A. Statistical and Machine Learning Results

The experimental results show that the proposed hybrid analysis layer successfully detected abnormal infrastructure behavior using both statistical methods and machine learning.

For CPU-related anomalies, repeated decreases in airflow-apiserver container CPU usage were detected against a historical baseline of approximately 0.65–0.70. The strongest examples reached Z-scores of 10.25 and 9.94, with EWMA and Rate of Change also firing, producing composite anomaly scores of 1.0. Another CPU event recorded a Z-score of 3.92 and was independently flagged by Isolation Forest, while EWMA and Rate of Change remained negative.

These results indicate that the different detectors do not always respond identically. Statistical methods such as Z-score, EWMA, and Rate of Change are sensitive to deviation from historical baselines and short-term changes, whereas Isolation Forest can identify observations that are unusual even when some statistical detectors do not fire. This detector disagreement provides complementary evidence rather than relying on a single detection rule.

The strongest machine-learning-supported result was observed in the memory experiment. The airflow-worker container reached approximately 3.66 GB of memory compared with a baseline of approximately 1.61 GB, representing around 127% growth. The event produced a Z-score of 10.4397, while EWMA, Rate of Change, and Isolation Forest all fired simultaneously. This was the only event in the reported dataset where all four detectors agreed.

Another memory event involving airflow-scheduler reached approximately 403.8 MB compared with a baseline of approximately 290.6 MB, with a Z-score of 4.4634 and Isolation Forest also returning true. This further confirms that the machine learning detector participated in real metric processing rather than existing only as an offline component.

The experimental detector outputs can be summarized as follows:

Scenario	Metric / Component	Z-score	EWMA	Rate of Change	Isolation Forest	Anomaly Score
CPU anomaly	airflow-apiserver CPU	10.25	Fired	Fired	N/R	1.0
CPU anomaly	airflow-apiserver CPU	3.92	False	False	True	N/R
Memory anomaly	airflow-worker memory	10.4397	Fired	Fired	Fired	0.9317
Memory anomaly	airflow-scheduler memory	4.4634	N/R	N/R	True	0.8093

The results demonstrate that the hybrid detector can capture different abnormal behaviors through complementary detection mechanisms. Statistical methods provide interpretable deviation measurements, while Isolation Forest contributes an additional machine learning perspective for identifying unusual observations.

A limitation of the current implementation is that the AIOps interface exposes Isolation Forest mainly as a boolean anomaly decision rather than a numeric Isolation Forest score. Z-score and Rate of Change provide numeric values, while EWMA is only partially exposed numerically.

Therefore, the comparison in this study focuses primarily on detector agreement and final anomaly decisions rather than direct numeric score comparison across all four detectors.

B. Incident Correlation and RCA Results

Experimental results demonstrate that the Event Correlation Engine is capable of aggregating operational events from multiple sources into incidents, based on host identity and temporal proximity. Within the queried data range, the AIOps Agent returned at least 20 open incidents, while another reporting query identified up to 50 incidents over the same period. The correlation score for many of these incidents is approximately 0.7.

Incidents are formed from various types of data, including metric anomalies related to container CPU and memory, log anomalies concerning SSH authentication, and other operational signals. This demonstrates that the correlation layer is capable of integrating metric events and log events, rather than processing each signal in isolation.

However, the results also demonstrate that correlation does not imply causation. The current mechanism primarily relies on two conditions:

- events belonging to the same host, node4-coldstore.
- events occurring within a correlation window of approximately 2–10 minutes.

Host correlation is confirmed as the primary join key, whereas temporal correlation merely indicates that events occurred close to each other in time.

The correlation results are summarized as follows:

Relationship	Correlation Result	Interpretation
Events on the same node4-coldstore host	Yes	Host correlation
Metric and log events within the same time window	Yes	Temporal correlation
SSH failures → authentication logs	Direct evidence exists	Causal relationship
SSH failures → CPU/memory anomalies	Correlated	No causal evidence
Different components on the same host/time window	May be grouped	Weak correlation

A notable case occurred when a single incident contained both a strong memory anomaly and SSH authentication failures. The airflow-worker memory usage increased to approximately 3.66 GB, with a Z-score of 10.44, while all four detectors were triggered. An SSH authentication burst occurred near the same period. Nevertheless, the RCA result concluded that these were two independent anomaly classes and that no evidence linked the SSH authentication failures to the container memory anomaly.

C. End-to-End AIOps Discussion

The end-to-end evaluation demonstrates that the proposed framework is able to connect infrastructure telemetry, statistical and machine learning analysis, event correlation, incident generation, and operator interaction within a single AIOps workflow.

The processing flow begins with metrics and logs collected from the monitored infrastructure. Prometheus and Grafana Loki provide the telemetry input, while the statistical and machine learning layer analyzes metric behavior using Z-score, EWMA, Rate of Change, and Isolation Forest. Detected abnormal conditions are converted into operational events and stored in the AIOps Operational Data Store.

These operational events are then processed by the Event Correlation Engine. Instead of presenting every metric or log anomaly independently, the correlation layer groups events that share common host and temporal characteristics into higher-level incidents. This reduces the amount of isolated information that must be inspected manually by an operator.

The experimental results also show that correlation alone is not sufficient to determine the actual cause of an incident. For example, SSH authentication failures and container resource anomalies were sometimes grouped into the same incident because they occurred on the same host within a similar time window. RCA analysis was therefore used to distinguish between simple co-occurrence and actual causal evidence. In the evaluated case, the SSH activity directly explained the authentication-failure logs, but no evidence showed that it caused the CPU or memory anomalies.

The final stage of the workflow exposes these results through the AIOps Tool/API Layer and the DeepSeek AIOps Agent. The operator can query the system using natural language to inspect abnormal metrics, logs, detector outputs, incidents, and RCA evidence without manually querying each monitoring backend or database.

The complete workflow therefore begins with telemetry, continues through statistical and machine learning analysis, operational event generation, event correlation, incident generation, and RCA, and concludes with operator interaction through the AIOps Agent.

From an operational perspective, the framework provides three main benefits. First, it combines multiple detection methods to provide stronger evidence of abnormal infrastructure behavior. Second, it reduces event fragmentation by correlating related signals into incidents. Third, it provides a unified interface through which operators can inspect infrastructure conditions and supporting evidence.

At the same time, the experiments reveal several limitations. The current correlation mechanism relies heavily on host identity and temporal proximity, which may group unrelated events into the same incident. In addition, some detector details, such as numeric Isolation Forest scores, are not fully exposed through the current AIOps interface. These limitations suggest that future improvements should focus on richer contextual correlation, component-level matching, and more complete detector evidence in incident records.

Overall, the end-to-end experiment shows that the proposed framework is not limited to isolated anomaly detection. Instead, it provides a complete operational pipeline that transforms raw infrastructure telemetry into structured events, correlated incidents, analytical evidence, and operator-facing information.

VI. CONCLUSION

This study presented a hybrid AIOps framework that combines statistical methods, machine learning, event correlation, RCA, and an operator-facing AIOps Agent for infrastructure monitoring and incident analysis. The framework was deployed on the infrastructure of a quantitative trading system, where real operational metrics and logs from services such as Apache Airflow were collected and analyzed.

The experimental results show that combining Z-score, EWMA, Rate of Change, and Isolation Forest provides complementary evidence for abnormal infrastructure behavior. Statistical methods were effective in identifying deviations from historical baselines, while Isolation Forest contributed an additional machine learning perspective for identifying unusual observations. The strongest observed memory anomaly on airflow-worker was detected by all four methods, demonstrating that the hybrid approach can provide stronger evidence than relying on a single detector.

The Event Correlation Engine further reduced isolated metric and log events by grouping them into higher-level incidents. This improves operational visibility because operators no longer need to inspect every anomaly individually. The experimental results also demonstrated an important distinction between correlation and causation. Events occurring on the same host within a similar time window can be correlated, but they do not necessarily share the same root cause. RCA was useful in verifying this distinction by showing, for example, that SSH authentication failures were directly supported by log evidence, while no causal relationship was established between SSH activity and CPU or memory anomalies.

The end-to-end experiment also demonstrated the practical value of the AIOps Agent. Instead of manually querying Prometheus, Grafana Loki, PostgreSQL, incidents, and RCA results separately, an operator can request an operational summary using natural language. This provides a more accessible interface for infrastructure inspection, incident review, and evidence analysis.

The main strengths of the proposed framework can be summarized as follows:

- It combines statistical and machine learning detectors instead of depending on a single detection method.
- It analyzes real operational telemetry generated by running infrastructure services.
- It integrates metric and log events into a common operational event model.
- It reduces fragmented alerts through event correlation and incident generation.
- It uses RCA to distinguish correlated evidence from causal evidence.
- It provides an end-to-end interface through the AIOps Agent for operator-oriented analysis.

However, the experiments also revealed several limitations. The current event correlation mechanism relies heavily on host identity and temporal proximity. As a result, unrelated metric and log events may be grouped into the same incident when they occur on the same node within a similar time period. In addition, component or service consistency is not yet sufficiently enforced during correlation.

The detector evidence exposed through the current AIOps interface is also incomplete. Z-score and Rate of Change provide numeric values, while EWMA information is only partially available and Isolation Forest is mainly exposed as a boolean decision rather than a numeric machine learning score. This limits direct quantitative comparison among the four detectors.

The Agent experiment also showed that data accessibility and evidence consistency are important. Some incident and RCA records do not expose all detector outputs, and truncated query results can prevent the Agent from presenting a complete operational picture. Therefore, the quality of the Agent response depends not only on the language model, but also on the completeness and consistency of the underlying AIOps data.

Future improvements should therefore focus on several directions. First, event correlation should incorporate component-level and service-level matching, together with tighter and adaptive correlation windows. Second, detector outputs should be stored and exposed consistently, including Z-score, EWMA deviation, Rate of Change, Isolation Forest decision score, thresholds, and relevant baseline information. Third, incident records should preserve richer evidence linking each correlated event to its detector outputs and source telemetry. Finally, the AIOps Agent should be provided with more structured and complete APIs so that it can retrieve operational evidence directly without relying on partial summaries or reconstructed context.

Overall, the experimental results indicate that the proposed framework provides a practical foundation for intelligent infrastructure operations. By combining statistical analysis, machine learning, event correlation, RCA, and natural-language interaction, the system can transform raw telemetry into structured operational knowledge that is more useful for monitoring, diagnosis, and incident investigation. At the same time, the observed limitations provide clear directions for improving correlation accuracy, detector transparency, and the reliability of agent-assisted infrastructure analysis.

VII. REFERENCES

[1] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation Forest," in *Proceedings of the 2008 Eighth IEEE International Conference on Data Mining (ICDM)*, Pisa, Italy, 2008, pp. 413–422, doi: 10.1109/ICDM.2008.17. Isolation Forest.

[2] National Institute of Standards and Technology, "EWMA Control Charts," *NIST/SEMATECH e-Handbook of Statistical Methods*. The EWMA statistic uses exponentially decreasing weights on historical observations and is suitable for detecting gradual shifts in a process. NIST.

[3] Prometheus Authors, "Prometheus: Monitoring system and time series database," *Prometheus Documentation*. Prometheus provides a dimensional time-series data model, PromQL, metric collection, and alerting capabilities. Prometheus.

[4] Grafana Labs, "Grafana Loki Overview," *Grafana Loki Documentation*. Loki provides scalable log aggregation and organizes log streams through labels while supporting query and analysis through LogQL. Grafana Labs.

[5] Grafana Labs, "Grafana Alloy Documentation," *Grafana Alloy Documentation*. Grafana Alloy is an open-source telemetry collector supporting metrics, logs, traces, and profiles, and can forward telemetry to backends such as Prometheus and Loki. Grafana Alloy.

[6] PostgreSQL Global Development Group, "JSON Types," *PostgreSQL Documentation*. PostgreSQL supports `json` and `jsonb` data types for storing and querying structured JSON data. PostgreSQL.